

Context-Optimized Multi-Agent Orchestration: A Reproducibility Review and Runtime Evaluation

Context Optimized Agents Project

May 23, 2026

Abstract

This paper evaluates `context_optimized_agents`, a Python implementation of a scoped-context multi-agent orchestration pattern. The system routes a user objective into role-specific context packets, applies token budgets, audits packets for redundant context, executes 35 specialized agents, compresses findings through parent agents, and records a final decision-memory artifact. We reviewed the implementation, ran its local verification gates, executed the deterministic workflow, generated its metrics and architecture diagram, and tested the OpenAI live integration with `gpt-5.5`.

The deterministic workflow completed successfully with 35 agent results, 7 parent results, 35 audited packets, no over-budget packets, a quality score of 0.91, and a hallucination-exposure score of 0.18. The included simulator reports a 74.1% input-token reduction, 56.5% output-token reduction, 68.6% estimated token-cost reduction, and 34.3% latency reduction versus a brute-force broadcast baseline. These metrics remain simulated planning estimates rather than measured production benchmarks. Live OpenAI access was valid, but the full live workflow failed because strict JSON responses were truncated at `max_output_tokens` and parsed without checking `response.status` or `incomplete_details`. This is the primary remaining blocker for production use.

1 Research Questions

1. Does the repository implement the advertised context-optimized orchestration topology?
2. Does the runnable workflow complete under the declared local test path?
3. Are the reported metrics measured, simulated, internally consistent, and reproducible?
4. What remains before the project can be treated as production-grade live OpenAI orchestration?

2 Methods

2.1 Repository and Environment

The project was evaluated from `/Users/Oxnexis/Desktop/context_optimized_agents` on May 23, 2026. The default `python3` interpreter was Python 3.9.6, below the package requirement of Python `>=3.10`, so a Python 3.11.14 virtual environment was created with:

```
/opt/homebrew/bin/python3.11 -m venv .venv  
.venv/bin/python -m pip install -e '[all,dev]'
```

2.2 Documentation and API Check

Official OpenAI developer documentation was consulted for current Responses API, Structured Outputs, and `gpt-5.5` behavior. The docs identify `gpt-5.5` as a current model ID, describe the Responses API as the recommended API for new projects, and document structured outputs through `text.format` JSON schema configuration.

2.3 Verification Commands

The following gates were run:

```
.venv/bin/ruff check .
.venv/bin/python -m compileall -q src tests
.venv/bin/python -m unittest discover -s tests
.venv/bin/pytest -q
.venv/bin/python -m context_optimized_agents run --mode mock \
  --objective-file examples/task_objective.txt \
  --output runs/mock_workflow_result.json --print-final
.venv/bin/python -m context_optimized_agents metrics --output runs/metrics.json
.venv/bin/python -m context_optimized_agents diagram --output-dir docs/figures
```

OpenAI live mode was tested in three stages: a minimal `gpt-5.5` Responses API smoke test, the default live CLI workflow, and a controlled live workflow with larger output budgets without changing repository code. The API key was supplied only through a silent stdin prompt into `OPENAI_API_KEY`; no secret was written to the repository.

3 System Description

The implemented architecture has the following main modules.

Module	Observed role
<code>ContextRouter</code>	Classifies the task and creates 35 role-specific context packets.
<code>BudgetAllocator</code>	Assigns fixed input and output token budgets per agent kind.
<code>ContextAuditor</code>	Deduplicates repeated retrieved memory and estimates packet-token savings.
<code>SpecializedAgent</code>	Calls the configured LLM client for each scoped packet.
<code>ParentAgent</code>	Compresses child findings per parent group.
<code>OrchestrationAgent</code>	Coordinates routing, audit, child execution, parent compression, final synthesis, metrics, and decision memory.
<code>OpenAIResponsesClient</code>	Calls the OpenAI Responses API with strict JSON Schema output.

The local workflow uses a deterministic `MockLLMClient`; live mode uses OpenAI Responses API calls.

4 Results

4.1 Local Quality Gates

Context-Optimized Multi-Agent Orchestration with Budgeted Context Routing

A scoped-context architecture for reducing token duplication, hallucination exposure, and synthesis overhead

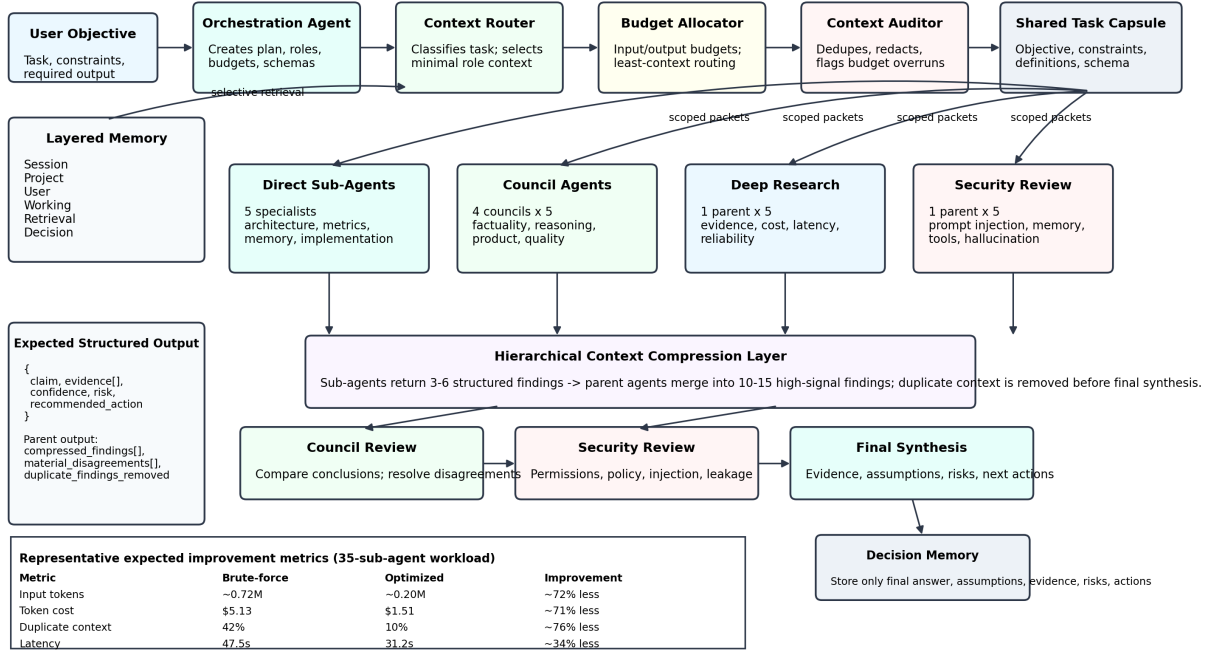


Figure 1. Scoped context routing uses shared task capsules, private role packets, strict budgets, progressive retrieval, and hierarchical compression before final synthesis.

Figure 1: Scoped-context multi-agent orchestration architecture.

Gate	Result	Evidence
Python requirement	Pass	Default Python 3.9.6 was unsuitable; <code>.venv</code> used Python 3.11.14.
Dependency install	Pass	<code>.[all,dev]</code> installed successfully.
Bytecode compilation	Pass	<code>compileall -q src tests</code> exited 0.
Unit tests	Pass	<code>unittest</code> : 4 tests passed.
Pytest	Pass	<code>pytest -q</code> : 4 tests passed.
Ruff lint	Pass after cleanup	25 fixable F401 unused-import violations were removed before repository push.
CodeRabbit review	Blocked	CLI authenticated, but the directory was not a git repository during the original review pass.

4.2 Deterministic Workflow Run

The mock-mode run completed and wrote `runs/mock_workflow_result.json`.

Measurement	Value
Agent results	35
Parent results	7
Audited packets	35
Estimated input tokens before audit	30,451

Estimated input tokens after audit	15,513
Duplicate-context tokens removed	17,554
Over-budget packets	0
Audit warnings	0
Final quality score	0.91
Hallucination-exposure score	0.18

The result confirms the deterministic path can execute end-to-end, produce structured findings, perform parent compression, and create a decision-memory record.

4.3 Simulated Metrics

The metrics command wrote `runs/metrics.json`. These values are explicitly simulated for a representative workload, not measured from production traces.

Metric	Brute-force	Context-optimized	Improvement
Input tokens	701,000	181,700	74.1% less
Output tokens	53,600	23,300	56.5% less
Estimated token cost	\$5.113	\$1.6075	68.6% less
Duplicate-context ratio	0.42	0.10	76.2% less
Mean prompt-to-final latency	47.5 s	31.2 s	34.3% less
Hallucination-exposure score	0.34	0.18	47.1% lower
Relevant-context density	0.46	0.81	76.1% higher

4.4 OpenAI Live Results

The `gpt-5.5` smoke test succeeded and returned the expected output. This confirms network access, credential validity, and model access.

The default live CLI workflow failed before writing `runs/live_workflow_result.json`. The failure occurred while parsing `response.output_text` as JSON:

```
json.decoder.JSONDecodeError: Unterminated string starting at:
line 1 column 3258
```

A single-agent probe reproduced the failure and showed the cause.

Field	Value
Response status	<code>incomplete</code>
Incomplete reason	<code>max_output_tokens</code>
Input tokens	1,144
Output tokens	600
Reasoning tokens	35
Output text length	3,248 characters
Parse result	<code>JSONDecodeError</code>

The direct-sub-agent output budget is 600 tokens, while the strict JSON schema can require more than that when the model returns several full findings and issues. The client does not check

`response.status`, `incomplete_details`, or refusal content before attempting `json.loads`. A controlled live run with expanded budgets progressed past the child-agent stage but later failed in parent synthesis with the same truncated JSON pattern at line 1 column 5721.

5 Findings: What Is Missing and What Remains

5.1 Live OpenAI Mode Is Not Production-Ready

The most important missing piece is robust handling for incomplete structured responses. The live client sends strict JSON schema requests, then parses `response.output_text` directly in `generate_json`. It should first check `response.status`, `incomplete_details`, refusal content, SDK parse helpers where available, and retryability or budget escalation policy.

5.2 Topology and Metrics Disagree

Documentation and metrics describe 6 parent agents, but the actual mock result contains 7 parent groups:

```
council_factuality
council_product
council_quality
council_reasoning
deep_research
direct_subagents
security_review
```

The implementation creates one parent for `direct_subagents`, one for `deep_research`, one for `security_review`, and one for each of four council groups. That is 7 parent agents, not 6. The metrics simulator assumes `parent_agents = 6`, and tests only assert `len(result.parent_results) >= 6`, which masks the mismatch.

5.3 Metrics Are Useful but Not Empirical Benchmarks

The simulator is transparent and reproducible, but it is not a measured benchmark. Claims about latency, duplicate-context ratio, hallucination exposure, relevant-context density, and cost should remain labeled as simulated until a harness measures actual API input/output tokens, cached-token effects, retry and failure rates, wall-clock latency, a brute-force baseline, and measured answer quality.

5.4 Static Gates and Test Coverage Need Expansion

The pushed repository now passes Ruff after cleanup, but it still declares `py.typed` without a configured strict type checker such as `mypy` or `pyright`. The current tests confirm deterministic happy paths only. Missing tests include exact parent topology, live-client `status=incomplete` handling, refusal handling, schema budget sizing, CLI failure modes, retry/backoff behavior, and partial artifact persistence.

5.5 Failure Isolation and Observability Are Underdeveloped

The live workflow uses `asyncio.gather` in a way that fails the whole run on a single agent or parent parse error. Production orchestration should preserve per-agent status, usage, timings, raw response metadata, and recoverable partial outputs. It should also provide progress reporting and structured error summaries.

6 Discussion

The project successfully demonstrates the intended local pattern: scoped context packets, role-specific budgets, audit-time deduplication, hierarchical compression, and decision-memory output. The deterministic run proves the control flow is coherent under mocked LLM behavior.

The live runs reveal the boundary between a convincing simulation and a production LLM system. Strict JSON schema output is the right direction, but strict schema does not remove the need to handle incomplete responses, refusals, budget exhaustion, parse failures, retries, and usage telemetry. The current implementation assumes that `output_text` is present and parseable. That assumption is false in ordinary live operation when `max_output_tokens` is reached.

The parent-agent count mismatch is also material. A system evaluating context efficiency must align its documented topology, runtime topology, tests, and metrics model. Otherwise, the token and cost results are internally inconsistent even when the simulator is deterministic.

7 Recommendations

1. Replace direct `json.loads(response.output_text)` with a response-normalization layer that handles completed, incomplete, refused, and malformed responses explicitly.
2. Recalibrate output budgets against each schema using live probes, then add regression tests that fail if schema-conformant outputs no longer fit.
3. Decide whether the intended topology has 6 or 7 parent agents, then update implementation, docs, metrics, and tests to match.
4. Add measured benchmark mode that records API usage, latency, success rate, parse retries, and token cost for both brute-force and optimized variants.
5. Add a strict type checker because the package ships `py.typed`.
6. Persist partial live-run artifacts so one failed agent does not erase all completed evidence.
7. Rotate any API key used for live evaluation if it was shared outside a local secret manager.

8 Conclusion

`context_optimized_agents` is a coherent deterministic prototype for scoped-context multi-agent orchestration. Its local run and simulated metrics support the architectural claim that role-specific routing and hierarchical compression can reduce context volume. However, the project is not yet production-grade live orchestration. The remaining work is concentrated in live response handling, budget calibration, topology consistency, empirical benchmarking, type gates, and failure observability.

9 Reproducibility Artifacts

Generated artifacts from this evaluation:

- `runs/mock_workflow_result.json`
- `runs/metrics.json`
- `docs/figures/context_orchestrator_academic_diagram.svg`
- `docs/figures/context_orchestrator_academic_diagram.png`
- `docs/figures/context_orchestrator_academic_diagram.pdf`

References

- [1] OpenAI. Responses API Overview. <https://developers.openai.com/api/reference/responses/overview>
- [2] OpenAI. Migrate to the Responses API. <https://developers.openai.com/api/docs/guides/migrate-to-responses>
- [3] OpenAI. Structured model outputs. <https://developers.openai.com/api/docs/guides/structured-outputs>
- [4] OpenAI. GPT-5.5 model documentation. <https://developers.openai.com/api/docs/models/gpt-5.5>
- [5] OpenAI. Using GPT-5.5. <https://developers.openai.com/api/docs/guides/latest-model>